TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

Lab 5: The File System with Your Own Hands

Operating Systems

Student: Gafenco Victor

Group: FAF-241

Instructor: Assistant lecturer, Patricia Reițman

Chișinău - 2026

CONTENTS

Abstract **3**

Introduction **4**

1 The tree and file types **5**

1.1 Recognizing objects 5

1.2 Comparing allocated space 5

2 Inodes and links **6**

2.1 Names and shared identity 6

2.2 Symbolic links store a pathname 6

2.3 Directory link counts: prediction and observation 7

2.4 Logical length and allocated storage 7

3 Permissions **8**

3.1 Modes and ownership 8

3.2 The creation mask 8

3.3 Reading and searching a directory 9

3.4 Set-user-ID, the sticky bit and protected files 9

4 Disks and the kernel as files **10**

4.1 Disk layout and root usage 10

4.2 CPU, memory and the meaning of self 10

5 Optional experiments **12**

5.1 File signatures rather than suffixes 12

5.2 Removing a file that remains open 12

Conclusions **13**

ABSTRACT

This laboratory investigates Linux file types, inode identity, links, permissions and storage. Experiments on openSUSE Tumbleweed establish the difference between names and file contents, demonstrate directory search permissions and sparse files, and map the root filesystem and live process information. The observed Btrfs directory link count is compared with the traditional Unix model.

Keywords: Linux, Btrfs, inode, hard link, symbolic link, permissions, procfs

INTRODUCTION

This laboratory examines Linux file types, inode identity, hard and symbolic links, permissions, disk layout and the process information exposed by procfs. The experiments were performed on 12 October 2026 as the ordinary user `victor` on openSUSE Tumbleweed. A fresh directory, `/home/victor/os-lab5/run-018y4c78`, kept the file experiments separate from existing work. Its filesystem is Btrfs with a reported block size of 4096 bytes.

The following sections answer every Observe question in the handout. Command excerpts show the measured results; the complete transcripts are retained in `evidence/1-tree.txt` through `evidence/5-stretch.txt`. Two optional experiments were completed: recognizing file signatures and reading an unlinked file through an existing descriptor.

1 THE TREE AND FILE TYPES

1.1 Recognizing objects

The command `ls /` showed the usual top-level names, including `bin`, `boot`, `dev`, `etc`, `home`, `proc`, `tmp`, `usr` and `var`. The requested detailed listing was:

```
$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx.  1 root root    7 Mar  2  2026 /bin -> usr/bin
crw-rw-rw-.  1 root root 1, 3 Oct  6 10:15 /dev/null
drwxr-xr-x.  1 root root 3654 Oct 12 15:12 /etc
-rw-r--r--.  1 root root 2202 Oct  5 14:07 /etc/passwd
drwxrwxrwt. 73 root root 3900 Oct 12 16:10 /tmp
```

Figure 1.1 – File types in the Linux root hierarchy

The first character identifies `/etc` and `/tmp` as directories (`d`), `/etc/passwd` as a regular file (`-`), `/bin` as a symbolic link (`l`), and `/dev/null` as a character device (`c`). The trailing dot after the mode is additional security-metadata notation, not a file type or permission bit.

`/bin` stores the relative target `usr/bin`. Since the link is in `/`, it resolves to `/usr/bin`. `file /usr/bin/ls /etc/passwd /bin` identified an x86-64 ELF executable, ASCII text, and a symbolic link respectively.

A character-device example is `/dev/null`, with major/minor numbers 1 and 3. The block-device search `find /dev -maxdepth 2 -type b -ls` found `/dev/nvme0n1` (259, 0) and its partitions, including `/dev/nvme0n1p5` (259, 5). Their `b` mode prefix distinguishes block devices from character devices.

1.2 Comparing allocated space

```
$ du -sh /var/log 2>/dev/null
320M    /var/log
$ du -sh /usr 2>/dev/null
13G     /usr
```

Figure 1.2 – Allocated space under /var/log and /usr

The visible allocation under `/usr` is approximately 42 times that under `/var/log`: `13 * 1024 / 320 = 41.6`. The difference is about 12.7 GiB. These are rounded values and lower bounds for the accessible directory contents: errors were suppressed as instructed, and some contents were inaccessible to the ordinary user. `du` measures allocated space, not the sum of logical file lengths.

2 INODES AND LINKS

2.1 Names and shared identity

After `echo hello > notes.txt`, `stat notes.txt` reported size 6 bytes, inode **468755**, and **Links: 1**. `Inode` identifies the inode within its filesystem; `Links` counts hard links to it. The sixth byte is the newline written by `echo`.

```
$ ln notes.txt hard.txt
$ ls -li notes.txt hard.txt
468755 -rw-r--r--. 2 victor victor 6 Oct 12 16:10 hard.txt
468755 -rw-r--r--. 2 victor victor 6 Oct 12 16:10 notes.txt
$ rm notes.txt
$ cat hard.txt
hello
$ ls -li hard.txt
468755 -rw-r--r--. 1 victor victor 6 Oct 12 16:10 hard.txt
```

Figure 2.1 – Hard-link identity before and after unlinking

Both names refer to the same inode and the same data. Removing `notes.txt` removes that directory entry and reduces the link count from 2 to 1. It does not privilege the first name or erase the contents while `hard.txt` still refers to them. After the last name is removed, open references can still keep the file alive; the optional experiment below demonstrates this.

2.2 Symbolic links store a pathname

```
$ ln -s hard.txt soft.txt
$ ls -li hard.txt soft.txt
468755 -rw-r--r--. 1 victor victor 6 Oct 12 16:10 hard.txt
468756 lrwxrwxrwx. 1 victor victor 8 Oct 12 16:10 soft.txt -> hard.txt
$ cat soft.txt
hello
$ rm hard.txt
$ cat soft.txt
cat: soft.txt: No such file or directory
$ ls -l soft.txt
lrwxrwxrwx. 1 victor victor 8 Oct 12 16:10 soft.txt -> hard.txt
```

Figure 2.2 – Symbolic-link identity and target lookup

The symbolic link has its own inode, 468756. Its length is exactly **8 bytes**, the number of characters in `hard.txt`, with no stored newline counted in its length. The pathname is resolved relative to the directory containing the link. Removing `hard.txt` leaves a dangling link: the link itself exists, but its target no longer resolves. A symbolic link does not increase its target's hard-link count.

2.3 Directory link counts: prediction and observation

The handout's traditional prediction was 2 links for a new directory and 4 after adding two child directories. In that model, the parent's entry for `d` and `d/.` account for two, and `d/x/..` and `d/y/..` add two more.

The actual Btrfs result differs:

```
$ mkdir d; ls -ld d
drwxr-xr-x. 1 victor victor 0 Oct 12 16:10 d
$ mkdir d/x d/y; ls -ld d
drwxr-xr-x. 1 victor victor 4 Oct 12 16:10 d
$ stat -f -c 'work filesystem: %T, block size %S' .
work filesystem: btrfs, block size 4096
```

Figure 2.3 – Directory link counts on Btrfs

The link-count column remains **1**. The final `4` is the directory's size column, not its link count. Btrfs does not expose the traditional `2 + number of subdirectories` count here. Thus the handout explains the traditional directory model, while this laptop's measurement must be reported as 1 and 1.

2.4 Logical length and allocated storage

```
$ echo x > one.txt; ls -l one.txt; du -h one.txt
-rw-r--r--. 1 victor victor 2 Oct 12 16:10 one.txt
4.0K    one.txt
$ truncate -s 1G big.img; ls -lh big.img; du -h big.img
-rw-r--r--. 1 victor victor 1.0G Oct 12 16:10 big.img
0       big.img
```

Figure 2.4 – Logical size and allocated space

`one.txt` contains `x` and a newline but its reported allocation occupies a 4096-byte unit. The sparse file has a logical length of 1 GiB (1,073,741,824 bytes); `truncate` extends it with holes rather than writing data blocks. Reading those holes yields zero bytes. `du` reports zero allocated file-data space at this point; filesystem metadata still exists. `rm big.img` removed the experimental file.

3 PERMISSIONS

3.1 Modes and ownership

The nine permission characters below omit the separate file-type character.

Octal	Permissions	Example use
600	`rw-------`	A private configuration file
644	`rw-r--r--`	A document others may read
755	`rwxr-xr-x`	A public directory or executable
700	`rwx------`	A private directory

```
$ echo secret > s.txt
$ chmod 600 s.txt; ls -l s.txt
-rw-------. 1 victor victor 7 Oct 12 16:10 s.txt
$ chmod 000 s.txt; cat s.txt
cat: s.txt: Permission denied
$ chmod u+r s.txt; cat s.txt
secret
```

Figure 3.1 – Read access under different file modes

An ordinary owner’s access is checked against the owner permission bits. Ownership permits changing the mode, but it does not itself grant read access when those bits are zero. Restoring owner read permission allows the file to be read again.

3.2 The creation mask

```
$ umask
0022
$ touch new.txt; mkdir newdir; ls -ld new.txt newdir
-rw-r--r--. 1 victor victor 0 Oct 12 16:10 new.txt
drwxr-xr-x. 1 victor victor 0 Oct 12 16:10 newdir
```

Figure 3.2 – Creation modes under umask 0022

With no overriding default ACL, the creation mode is the requested mode with the mask bits cleared. A normal file starts from `0666`, producing `0644` under `0022`; a directory starts from `0777`, producing `0755`. The operation is a bit mask, not general arithmetic subtraction. A new ordinary file does not acquire execute bits merely because the mask allows them.

3.3 Reading and searching a directory

Before each test, the prediction distinguished listing names from looking up a known name. The file `box/f` was created with `echo hi > box/f`.

```
$ chmod u=r,go= box; ls box; cat box/f
f
cat: box/f: Permission denied
$ chmod u=x,go= box; ls box; cat box/f
ls: cannot open directory 'box': Permission denied
hi
$ chmod 755 box
```

Figure 3.3 – Directory read and search permissions

Directory **read** permission permits enumerating entry names. Directory **execute**, also called search permission, permits traversing it and resolving a known entry. Consequently a read-only directory can reveal `f` without allowing it to be opened; a search-only directory allows `box/f` to be opened when its name is known and the file's own mode permits reading, but prevents listing. Directory write permission, normally combined with search, controls creating and removing entries.

3.4 Set-user-ID, the sticky bit and protected files

```
$ ls -l /usr/bin/passwd
-rwsr-xr-x. 1 root shadow 97408 Aug 10 09:57 /usr/bin/passwd
$ ls -ld /tmp
drwxrwxrwt. 77 root root 3980 Oct 12 16:10 /tmp
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
$ ls -l /etc/shadow
-rw-r-----. 1 root shadow 977 Oct 5 14:07 /etc/shadow
```

Figure 3.4 – Set-user-ID, sticky bit and protected-file access

The denied read was recorded in an earlier check as the same ordinary user. The final run also confirmed that the file was not readable.

The `s` occupies the owner's execute position: owner execution and the set-user-ID bit are both set. Under normal execution conditions, `passwd` runs with its owner's effective user ID, root, allowing controlled updates to protected password data after its own checks. The user cannot directly read `/etc/shadow`; its mode grants access only to root and the shadow group.

The `t` in `/tmp` means other-execute plus the sticky bit. Even though many users can create entries in this directory, removal or renaming of someone else's entries is restricted to the file owner, directory owner or a suitably privileged process. It does not make file contents private; file modes still govern that.

4 DISKS AND THE KERNEL AS FILES

4.1 Disk layout and root usage

`lsblk -o NAME,TYPE,FSTYPE,SIZE,MOUNTPOINTS` established the following layout. All partitions belong to the same 931.5 GiB NVMe disk; repeated Btrfs mountpoints are not additional disks.

```
nvme0n1                 931.5G  disk
|-- nvme0n1p1   vfat      200M  /boot/efi
|-- nvme0n1p2             16M   no mounted filesystem shown
|-- nvme0n1p3   ntfs    243.3G  not mounted
|-- nvme0n1p4   ntfs      846M  not mounted
|-- nvme0n1p5   btrfs   685.2G  /, /home, /var, ...
`-- nvme0n1p6   swap        2G  [SWAP]
```

Figure 4.1 – NVMe disk partitions and mount points

The root mount is `/dev/nvme0n1p5`, filesystem type **Btrfs**, with `rw` in its mount options. The selected root subvolume is `/@/.snapshots/40/snapshot`, subvolume ID 303. The `df -h` row was:

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p5  686G   80G  605G  12% /
```

Figure 4.2 – Root filesystem usage

Thus root was **12% full**, using approximately 80 GiB, with 605 GiB available. Human-readable rounding explains why the displayed sizes need not add exactly.

4.2 CPU, memory and the meaning of self

```
$ grep -c processor /proc/cpuinfo
8
$ head -3 /proc/meminfo
MemTotal:       16212804 kB
MemFree:         1033936 kB
MemAvailable:    7479348 kB
$ head -5 /proc/self/status
Name:   head
Umask:  0022
State:  R (running)
Tgid:   842397
Ngid:   0
```

Figure 4.3 – CPU, memory and process information

The processor count establishes **8 logical CPUs**, not necessarily eight physical cores; simultaneous multithreading can expose multiple logical CPUs per core. The kernel reports **16,212,804 KiB**, approxi-

mately **15.46 GiB**, of total usable RAM. This is distinct from currently free or available memory and may be slightly below the installed module capacity.

The process name is **head** because `/proc/self` resolves for the process accessing the pathname. Here `head` opens and reads its own status, rather than the shell's status. Similarly, `ls -l /proc/self/fd` listed the descriptors of `ls`: 0, 1 and 2 referred to pipes in this run, and descriptor 3 referred to `/proc/842398/fd`, opened while listing that directory. These are live kernel views, not ordinary stored files.

5 OPTIONAL EXPERIMENTS

5.1 File signatures rather than suffixes

```
$ gzip -c /etc/passwd > notes.txt
$ file notes.txt
notes.txt: gzip compressed data, was "passwd", ...
$ xxd notes.txt | head -1
00000000: 1f8b 0808 f584 c36a 0003 7061 7373 7764  .......j..passwd
```

Figure 5.1 – Gzip identification from file contents

The first two bytes, **1f 8b**, are the gzip signature. The next byte, `08`, denotes the DEFLATE compression method. The `.txt` suffix does not change the content: `file` recognizes the header. The ellipsis shortens the descriptive timestamp fields; the complete output is in the transcript.

5.2 Removing a file that remains open

```
$ printf 'open file remains readable\n' > bigfile
$ exec 3< bigfile
$ rm bigfile
$ ls -l /proc/$$/fd
# Descriptor 3 points to:
# /home/victor/os-lab5/run-018y4c78/bigfile (deleted)
$ cat <&3
open file remains readable
$ exec 3<&-
```

Figure 5.2 – Reading an unlinked file through an open descriptor

The shell expands `$$` to its own process ID, so this time the directory describes the shell’s descriptors. The link ends in **(deleted)** because the name has been unlinked, but descriptor 3 still holds the open file. Its contents remain readable. Closing descriptor 3 releases that reference; with no links or other open references remaining, the file’s storage becomes reclaimable. On filesystems with snapshots or shared extents, physical space can remain referenced elsewhere. The subsequent descriptor listing contained only 0, 1 and 2.

CONCLUSIONS

The experiments separate names from inode identity, logical file length from allocated storage, and file permissions from directory search permissions. Hard links preserve access to the same inode after one name is removed, while symbolic links can remain present but dangling. The Btrfs directory-link result also shows why measured output must take precedence over a generic filesystem example. Disk and procfs inspection connected these abstractions to the laptop’s root partition, logical CPUs, memory and live processes.